

# Database Schema Design — v3

Return & Chargeback Risk Scorer — final

The final schema. v1 defined six tables; the 65-endpoint API needed thirty. This revision corrects four Razorpay fidelity errors verified against Razorpay's live entity documentation, closes a target-leakage path that would have inflated the held-out metrics without any visible symptom, and — new in v3 — binds the design to the chosen training dataset, recording which tables load, which stay deliberately empty, and which planned features have no source.

TABLES

**30** (v1: 6)

INDEXES

**36** (v1: 0)

ENGINES

**SQLite 3.31+ · PostgreSQL 12+**

TIMESTAMPS

**INTEGER epoch**

AMOUNTS

**INTEGER paisa**

STATUS

**Final · DDL verified**

**The one change that matters most.** v1's customers table stored a single mutable `past_dispute_rate` annotated “as-of transaction time”, with nothing in the schema able to make as-of true. Joining to it at training time hands the model the customer's *final* return rate — which already contains the outcome being predicted. A chronological train/test split does not catch this, because the leak arrives through an aggregate that has no time dimension at all. v2 splits identity from statistics, stamps the aggregate with `as_of_ts`, bars it from training, and adds `risk_score_features` as the frozen as-of vector that training reads instead.

# 1 · What changed from v1, and why

Grouped by severity. Every item is closed by the DDL in Appendix A.

## Blocking The API could not run on v1

| #  | PROBLEM IN V1                                                                                                                                         | RESOLUTION IN V2                                                                                                 |
|----|-------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| B1 | Of 65 endpoints, only 11 had any table behind them. Scoring, explanations, reviews, models, evaluations, thresholds, ingestion and webhooks had none. | 24 tables added across six new layers. Coverage matrix in §8.                                                    |
| B2 | No merchant_id on any table, while the API asserts that every key is merchant-scoped and can never read another merchant's rows.                      | merchants and api_keys added; merchant_id NOT NULL on every scoped table; role on the key gates admin endpoints. |

## Correctness Would have produced wrong or flattering metrics

| #  | PROBLEM IN V1                                                                                                                                                                                   | RESOLUTION IN V2                                                                                                                                                                              |
|----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| C1 | <b>Target leakage.</b> customers held one mutable aggregate row; "as-of transaction time" was unenforceable. Training joins would import future outcomes, invisibly to the chronological split. | Identity split into customers; statistics into customer_stats_current carrying as_of_ts and documented as serving-only. Training reads risk_score_features or recomputes as-of (§6, Query 2). |
| C2 | <b>No label maturity.</b> A payment with no dispute row was indistinguishable from one whose dispute window was still open, so the model would learn "recent = safe".                           | risk_labels.label_matured_at, is_mature, outcome_observed_at, driven by merchants.dispute_window_days.                                                                                        |
| C3 | <b>Polluted positive class.</b> label_refunded counted every refund, including merchant-side ones (out of stock, pricing error) that are not return risk.                                       | refunds.refund_reason enum; only customer_return and customer_cancellation enter the label (§6, Query 1).                                                                                     |
| C4 | <b>Out-of-scope losses in the label.</b> disputes.phase omitted Razorpay's fraud and retrieval values — precisely the phases the project's scope excludes.                                      | Full five-value enum stored; both excluded at derivation and recorded in risk_labels.excluded_reason.                                                                                         |
| C5 | past_dispute_rate = total_disputes / total_orders, but disputes attach to payments and one order can carry several attempts.                                                                    | Denominator is total_payments_captured; total_returns added alongside total_refunds.                                                                                                          |
| C6 | No record of which rows were in which split, so a re-run silently re-split and the "frozen held-out set" was not frozen.                                                                        | datasets (with train_until and split_hash) and dataset_members; models and evaluations both reference a dataset.                                                                              |
| C7 | Scores were not reproducible: the aggregates a score was computed from would have moved by the time anyone asked why.                                                                           | risk_score_features freezes the exact vector and per-feature contribution at scoring time.                                                                                                    |

## Fidelity Deviations from Razorpay's published entities

| #  | PROBLEM IN V1                                                                                                                                      | RESOLUTION IN V2                                |
|----|----------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------|
| R1 | created_at declared TIMESTAMP. Razorpay returns created_at and respond_by as <b>integer Unix epoch</b> . This also contradicted the API reference. | All temporal columns are INTEGER epoch seconds. |

|    |                                                                                                                                                                  |                                                                                                                                                                  |
|----|------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| R2 | disputes.status listed four values; Razorpay defines five.                                                                                                       | closed added.                                                                                                                                                    |
| R3 | disputes lacked amount_deducted and reason_description; the DDL also dropped currency, which the field table listed.                                             | All three present. amount_deducted — what actually leaves the balance on a lost dispute — is the correct false-negative cost input.                              |
| R4 | <b>Field tables and SQL disagreed.</b> payments documented card_id/bank/wallet/vpa, fee, tax, error_code, error_description — the CREATE TABLE had none of them. | All restored, plus international, description, acquirer_data, error_source, error_step, error_reason.                                                            |
| R5 | method omitted emi. Separately, the API accepted cod, which is not a Razorpay method at all.                                                                     | emi added. cod retained — it is a first-order return-risk driver — but explicitly commented as a non-native extension rather than passed off as Razorpay-native. |
| R6 | refunds DDL dropped notes; entity also lacked speed_requested, receipt, batch_id.                                                                                | All present.                                                                                                                                                     |
| R7 | orders lacked offer_id.                                                                                                                                          | Added.                                                                                                                                                           |

#### Coverage

#### Documented features with nowhere to live

| #  | PROBLEM IN V1                                                                                                                                                                       | RESOLUTION IN V2                                                                     |
|----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| F1 | The entire product-level feature family — category baseline return rate, price relative to category average, size/fit dependency — was unimplementable. No products, no line items. | products and order_items, with size_variant_type on the product.                     |
| F2 | Velocity signals (ip, device_id, session_id) were accepted by the scoring endpoint and stored nowhere.                                                                              | payment_context, carrying an explicit scope guard against cross-account graph joins. |

#### Hygiene

| #  | PROBLEM IN V1                                                                                                                                                   | RESOLUTION IN V2                                                                                                          |
|----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| H1 | No indexes. Velocity features are per-customer range scans over created_at — full table scans without them.                                                     | 36 indexes, including three partial-unique constraints (one active model, one current config of each kind, per merchant). |
| H2 | No NOT NULL, no CHECK. Every status column was free text, so Captured and captured both inserted.                                                               | Every enum constrained; nullability declared throughout.                                                                  |
| H3 | customers keyed on a raw phone number — contradicting the cust_xxxx ids the API exposes, breaking on number change, and putting PII in a primary key and a URL. | Synthetic cust_id; email and contact become indexed attributes.                                                           |

## 2 · Conventions

| CONCERN     | RULE                                                                                                                 |
|-------------|----------------------------------------------------------------------------------------------------------------------|
| Timestamps  | INTEGER, Unix epoch seconds, matching Razorpay's created_at and respond_by. No TIMESTAMP columns anywhere.           |
| Amounts     | INTEGER in the smallest currency unit — paise for INR.                                                               |
| Identifiers | Prefixed strings: acc_key_cust_order_pay_rfnd_disp_prod_oitem_score_rev_usr_mdl_eval_ds_thr_cost_job_.               |
| Tenancy     | merchant_id NOT NULL on every scoped table. Never accepted as a request parameter — always derived from the API key. |
| Enums       | Enforced with CHECK constraints, not conventions. Nullable enums use IS NULL OR ... IN (...).                        |
| JSON        | Declared TEXT for SQLite; change to JSONB on PostgreSQL. Every such column is commented - - json.                    |
| Booleans    | BOOLEAN; SQLite stores 0/1, PostgreSQL natively.                                                                     |
| Ordering    | The DDL is dependency-ordered and runs top-to-bottom on both engines with no forward references.                     |
| Provenance  | Tables are tagged <b>RZP</b> where they mirror a Razorpay public entity and <b>EXT</b> where they are ours.          |

## 3 · Layer map — 30 tables

| LAYER                      | TABLES                                                                              | PURPOSE                                                     |
|----------------------------|-------------------------------------------------------------------------------------|-------------------------------------------------------------|
| <b>A</b> Tenancy           | merchants, api_keys                                                                 | Scoping and the admin-role gate.                            |
| <b>B</b> Identity          | customers                                                                           | Who the customer is. Deliberately holds no statistics.      |
| <b>C</b> Razorpay-native   | orders, payments, refunds, disputes                                                 | The transaction chain, mirroring Razorpay's entities.       |
| <b>D</b> Commerce detail   | products, order_items, payment_context                                              | Where the product-level and velocity features come from.    |
| <b>E</b> Aggregates        | customer_stats_current                                                              | Serving-time cache. Barred from training.                   |
| <b>F</b> Labels & datasets | risk_labels, datasets, dataset_members                                              | The target, its maturity, and the frozen split.             |
| <b>G</b> Models & eval     | models, model_feature_importance, cost_model_config, evaluations, evaluation_points | Candidates, held-out metrics, and the cost curve.           |
| <b>H</b> Thresholds        | threshold_config                                                                    | Versioned cut points, linked to the run that produced them. |
| <b>I</b> Scoring           | risk_scores, risk_score_features, risk_explanations                                 | Output, its frozen inputs, and the LLM's prose.             |
| <b>J</b> Review            | reviewers, reviews, review_decisions, review_notes                                  | The only path from a score to an action.                    |
| <b>K</b> Operations        | ingest_jobs, webhook_events, idempotency_keys                                       | Bulk load, inbound events, request replay safety.           |

## 4 · Razorpay-native entities

Field sets and enum values verified against Razorpay's published Payment and Dispute entity documentation.

### orders RZP

| FIELD                    | TYPE      | NOTES                                                                                                                                      |
|--------------------------|-----------|--------------------------------------------------------------------------------------------------------------------------------------------|
| id                       | TEXT PK   | order_XXXXXXXXXXXX                                                                                                                         |
| merchant_id              | TEXT FK   | <b>NEW</b> → merchants.id, NOT NULL                                                                                                        |
| amount                   | INTEGER   | Paise. CHECK >= 0                                                                                                                          |
| amount_paid / amount_due | INTEGER   | Default 0                                                                                                                                  |
| currency                 | TEXT      | Default INR                                                                                                                                |
| receipt                  | TEXT      | Merchant's internal reference                                                                                                              |
| status                   | TEXT      | created · attempted · paid — now CHECK-constrained                                                                                         |
| attempts                 | INTEGER   | Payment attempts; a velocity feature                                                                                                       |
| offer_id                 | TEXT      | <b>NEW</b> Razorpay field, missing in v1                                                                                                   |
| country                  | TEXT      | <b>v3</b> <b>EXT</b> Not a Razorpay Order field. Added because the training data carries a country per invoice and it is a usable feature. |
| notes                    | TEXT json | Merchant metadata                                                                                                                          |
| created_at               | INTEGER   | <b>FIXED</b> was TIMESTAMP; Razorpay returns epoch                                                                                         |

### payments RZP

| FIELD                                  | TYPE          | NOTES                                                                                                         |
|----------------------------------------|---------------|---------------------------------------------------------------------------------------------------------------|
| id                                     | TEXT PK       | pay_XXXXXXXXXXXX                                                                                              |
| merchant_id                            | TEXT FK       | <b>NEW</b> NOT NULL                                                                                           |
| order_id                               | TEXT FK       | → orders.id                                                                                                   |
| customer_id                            | TEXT FK       | <b>NEW</b> → customers.id, resolved at ingest. Replaces the implicit email/contact join.                      |
| amount, currency                       | INTEGER, TEXT | —                                                                                                             |
| status                                 | TEXT          | created · authorized · captured · refunded · failed                                                           |
| method                                 | TEXT          | <b>FIXED</b> card · netbanking · wallet · emi · upi (Razorpay) + cod (our extension, commented as non-native) |
| captured                               | BOOLEAN       | —                                                                                                             |
| international                          | BOOLEAN       | <b>NEW</b>                                                                                                    |
| description                            | TEXT          | <b>NEW</b>                                                                                                    |
| card_id, bank, wallet, vpa             | TEXT          | <b>FIXED</b> documented in v1 but absent from its DDL                                                         |
| acquirer_data                          | TEXT json     | <b>NEW</b>                                                                                                    |
| email, contact                         | TEXT          | Payer contact; indexed                                                                                        |
| fee, tax                               | INTEGER       | <b>FIXED</b> absent from v1 DDL                                                                               |
| error_code, error_description          | TEXT          | <b>FIXED</b> absent from v1 DDL                                                                               |
| error_source, error_step, error_reason | TEXT          | <b>NEW</b> Razorpay fields                                                                                    |
| amount_refunded                        | INTEGER       | Default 0                                                                                                     |
| refund_status                          | TEXT          | NULL · partial · full                                                                                         |
| notes                                  | TEXT json     | <b>NEW</b>                                                                                                    |

|            |         |                    |
|------------|---------|--------------------|
| created_at | INTEGER | <b>FIXED</b> epoch |
|------------|---------|--------------------|

## refunds RZP

| FIELD                   | TYPE          | NOTES                                                                 |
|-------------------------|---------------|-----------------------------------------------------------------------|
| id                      | TEXT PK       | rfnd_xxxxxxxxxxxx                                                     |
| merchant_id, payment_id | TEXT FK       | NOT NULL                                                              |
| amount, currency        | INTEGER, TEXT | —                                                                     |
| status                  | TEXT          | pending · processed · failed                                          |
| speed_requested         | TEXT          | <b>NEW</b> normal · optimum                                           |
| speed_processed         | TEXT          | normal · instant                                                      |
| receipt, batch_id       | TEXT          | <b>NEW</b>                                                            |
| notes                   | TEXT json     | <b>FIXED</b> documented in v1, absent from its DDL                    |
| refund_reason           | TEXT          | <b>NEW</b> <b>EXT</b> — see below. Razorpay carries no refund reason. |
| created_at              | INTEGER       | <b>FIXED</b> epoch                                                    |

**Why refund\_reason is not optional.** The label is returned OR disputed. Without a reason, a refund issued because the merchant was out of stock becomes a positive example of *customer* return risk, and the model learns to predict the merchant's own inventory problems. Only customer\_return and customer\_cancellation enter label\_refunded; merchant\_stock\_out, merchant\_error, duplicate\_charge and goodwill are recorded and excluded.

## disputes RZP

| FIELD                   | TYPE      | NOTES                                                                                                  |
|-------------------------|-----------|--------------------------------------------------------------------------------------------------------|
| id                      | TEXT PK   | disp_xxxxxxxxxxxx                                                                                      |
| merchant_id, payment_id | TEXT FK   | NOT NULL                                                                                               |
| amount                  | INTEGER   | Disputed amount                                                                                        |
| currency                | TEXT      | <b>FIXED</b> documented in v1, absent from its DDL                                                     |
| amount_deducted         | INTEGER   | <b>NEW</b> What actually leaves the balance when a dispute is lost — the correct FN cost input.        |
| reason_code             | TEXT      | NOT NULL                                                                                               |
| reason_description      | TEXT      | <b>NEW</b>                                                                                             |
| respond_by              | INTEGER   | <b>FIXED</b> epoch, not TIMESTAMP                                                                      |
| status                  | TEXT      | <b>FIXED</b> open · under_review · won · lost · closed — v1 omitted closed                             |
| phase                   | TEXT      | <b>FIXED</b> fraud · retrieval · chargeback · pre_arbitration · arbitration — v1 omitted the first two |
| evidence                | TEXT json | Submitted documents                                                                                    |
| created_at              | INTEGER   | <b>FIXED</b> epoch                                                                                     |

**The missing phases were a scope leak, not a typo.** This project's entire framing is that third-party fraud belongs to a different track and is not modelled. Razorpay marks exactly that distinction in phase: fraud is suspected third-party fraud, and retrieval is an information request rather than a loss at all. With those two values absent from v1's enum there was no way to filter them, so the training target would have quietly included the very class the write-up claims to exclude — and the scope argument made to judges would not have survived a look at the data.

## 5 · Customers, and the as-of rule

The correctness fix at the centre of this revision.

### customers EXT — identity only

| FIELD                       | TYPE    | NOTES                                                             |
|-----------------------------|---------|-------------------------------------------------------------------|
| id                          | TEXT PK | <b>FIXED</b> cust_XXXXXXXXXXXX. v1 keyed on the raw phone number. |
| merchant_id                 | TEXT FK | NOT NULL                                                          |
| email, contact              | TEXT    | Now indexed attributes, not the key. Either may be null.          |
| account_created_at          | INTEGER | Merchant's signup timestamp; drives the account-age feature       |
| first_seen_at, last_seen_at | INTEGER | <b>NEW</b> First and most recent observed transaction             |

### customer\_stats\_current EXT — serving cache, never a training source

| FIELD                   | TYPE    | NOTES                                                                                                           |
|-------------------------|---------|-----------------------------------------------------------------------------------------------------------------|
| customer_id             | TEXT PK | → customers.id                                                                                                  |
| as_of_ts                | INTEGER | <b>NEW</b> The moment these counters are true for. Exists so the row can never be mistaken for a timeless fact. |
| total_orders            | INTEGER | —                                                                                                               |
| total_payments_captured | INTEGER | <b>NEW</b> The correct rate denominator                                                                         |
| total_refunds           | INTEGER | All refunds                                                                                                     |
| total_returns           | INTEGER | <b>NEW</b> Refunds whose reason is customer-driven                                                              |
| total_disputes          | INTEGER | In-scope phases only                                                                                            |
| past_return_rate        | REAL    | <b>NEW</b> total_returns / total_payments_captured                                                              |
| past_dispute_rate       | REAL    | <b>FIXED</b> denominator was total_orders in v1                                                                 |

**Leakage rule, stated in the DDL itself.** Training and evaluation must not read this table. A row read at training time carries the customer's *final* return rate, which already contains the outcome of the payment being predicted plus every outcome after it. The chronological split does not protect you, because the split is on payments.created\_at while the leak arrives through an aggregate with no time dimension. The symptom is excellent held-out precision and recall that you would then present as honest. Training reads risk\_score\_features, or recomputes as-of values with Query 2 in §6.

## 6 · Labels, maturity and the frozen split

### risk\_labels EXT

| FIELD               | TYPE    | NOTES                                                                                            |
|---------------------|---------|--------------------------------------------------------------------------------------------------|
| payment_id          | TEXT PK | → payments.id                                                                                    |
| label_refunded      | BOOLEAN | <b>FIXED</b> customer-driven refunds only                                                        |
| label_disputed      | BOOLEAN | <b>FIXED</b> in-scope dispute phases only                                                        |
| label_risk          | BOOLEAN | The training target: label_refunded OR label_disputed                                            |
| outcome_observed_at | INTEGER | <b>NEW</b> When the outcome landed, if it has                                                    |
| label_matured_at    | INTEGER | <b>NEW</b> payment.created_at + dispute_window_days                                              |
| is_mature           | BOOLEAN | <b>NEW</b> Whether the dispute window has closed                                                 |
| excluded_reason     | TEXT    | <b>NEW</b> fraud_dispute · retrieval_request · merchant_refund · immature · payment_not_captured |

**Maturity is why `is_mature` exists.** A payment from last week with no dispute row is not *clean* — it is *unresolved*. Treating the two alike teaches the model that recent orders are safe, and since the held-out slice is the most recent data by construction, the error lands squarely on the metrics you report. `merchants.dispute_window_days` (default 120) sets the window; GET /risk-labels/coverage reads `is_mature` by cohort week.

#### REFERENCE QUERY 1 — LABEL DERIVATION, WITH BOTH EXCLUSIONS

```
SELECT
  p.id AS payment_id, p.merchant_id,
  -- customer-driven refunds only: merchant-side refunds are not return risk
  CASE WHEN EXISTS (
    SELECT 1 FROM refunds r
    WHERE r.payment_id = p.id AND r.status = 'processed'
      AND r.refund_reason IN ('customer_return','customer_cancellation')
  ) THEN 1 ELSE 0 END AS label_refunded,
  -- in-scope dispute phases only: 'fraud' and 'retrieval' are excluded
  CASE WHEN EXISTS (
    SELECT 1 FROM disputes d
    WHERE d.payment_id = p.id
      AND d.phase NOT IN ('fraud','retrieval')
      AND d.status IN ('open','under_review','lost','closed')
  ) THEN 1 ELSE 0 END AS label_disputed,
  p.created_at + (m.dispute_window_days * 86400) AS label_matured_at,
  CASE WHEN p.created_at + (m.dispute_window_days * 86400) <= strftime('%s','now')
    THEN 1 ELSE 0 END AS is_mature
FROM payments p JOIN merchants m ON m.id = p.merchant_id
WHERE p.captured = 1;
```

#### REFERENCE QUERY 2 — LEAKAGE-FREE AS-OF CUSTOMER FEATURES

```
SELECT
  p.id AS payment_id, p.created_at AS as_of_ts,
  COUNT(prior.id) AS prior_payments,
  SUM(CASE WHEN EXISTS (
    SELECT 1 FROM refunds r
    WHERE r.payment_id = prior.id AND r.created_at < p.created_at
      AND r.refund_reason IN ('customer_return','customer_cancellation')
  ) THEN 1 ELSE 0 END) AS prior_returns,
  SUM(CASE WHEN EXISTS (
    SELECT 1 FROM disputes d
    WHERE d.payment_id = prior.id AND d.created_at < p.created_at
      AND d.phase NOT IN ('fraud','retrieval')
  ) THEN 1 ELSE 0 END) AS prior_disputes
FROM payments p
LEFT JOIN payments prior
  ON prior.customer_id = p.customer_id
  AND prior.merchant_id = p.merchant_id
  AND prior.created_at < p.created_at -- the whole leakage fix is this line
  AND prior.captured = 1
GROUP BY p.id, p.created_at;
```

### datasets · dataset\_members EXT

`datasets` records `train_until` (the chronological cut), `exclude_immature`, row counts, the positive rate, and a `split_hash` over the member set. `dataset_members` pins each payment to train or test. Models and evaluations both reference a dataset id, so the API's promise that the held-out slice is written once and hashed is now enforceable rather than aspirational.



## 7 · Scoring, review, models and operations

Twenty-four tables that v1 did not have. Full column lists in Appendix A.

### Scoring EXT

| TABLE               | WHAT IT HOLDS, AND THE DESIGN DECISION IN IT                                                                                                                                                                                                           |
|---------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| risk_scores         | One row per score: probability, band, threshold_applied and the threshold_config_id it came from, recommendation, and customer_history. Storing the threshold config id means an old score can be re-read under the rules that actually applied to it. |
| risk_score_features | The frozen feature vector and per-feature contribution. Serves reproducibility, GET /risk/scores/{id}/features, and leakage-free training from one table.                                                                                              |
| risk_explanations   | The LLM's prose, its status, and risk_probability_at_generation — a copy of the score the text was written against, so a stale explanation is detectable. No column here can alter a score.                                                            |

### Review EXT

| TABLE            | WHAT IT HOLDS, AND THE DESIGN DECISION IN IT                                                                                                                                                                              |
|------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| reviewers        | People who may decide. A decision without a row here is impossible.                                                                                                                                                       |
| reviews          | Queue item opened when a score lands in a flagged band; open → assigned → closed.                                                                                                                                         |
| review_decisions | Append-only. reviewer_id NOT NULL — there is no system reviewer, and neither the model nor the LLM can write here. A reversal is a new row on a reopened review, never an update, so the audit trail cannot be rewritten. |
| review_notes     | Free-text notes that do not close the item.                                                                                                                                                                               |

### Models, evaluation and configuration EXT

| TABLE                    | WHAT IT HOLDS, AND THE DESIGN DECISION IN IT                                                                                                                                                                                                        |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| models                   | Algorithm, hyperparameters, ordered feature list, dataset, artifact path, lifecycle status. A partial unique index enforces exactly one active model per merchant.                                                                                  |
| model_feature_importance | Coefficient or gain per feature, ranked.                                                                                                                                                                                                            |
| evaluations              | Held-out run: precision, recall, F1, PR-AUC, ROC-AUC, base rate, confusion counts, total cost, optimal threshold. References the cost_model_id in force at run time, so an old run cannot be reinterpreted under new cost assumptions.              |
| evaluation_points        | One row per threshold step: counts, precision, recall, and total_cost. The series behind the cost-vs-threshold chart; optimal_threshold is its argmin, not the accuracy-maximising point.                                                           |
| cost_model_config        | FP and FN costs stored as <i>formula inputs</i> — lost-sale probability, AOV, trust penalty; refund amount, chargeback fee, restocking — not as bare numbers, so the figures can be audited. reason is NOT NULL and versions are never overwritten. |
| threshold_config         | Band cut points, source of manual or optimized, and the eval_id whose cost curve produced them. One current row per merchant.                                                                                                                       |

### Commerce detail & operations

| TABLE                 | WHAT IT HOLDS, AND THE DESIGN DECISION IN IT                                                                                                                                                                                                            |
|-----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| products, order_items | Category, list price and size_variant_type make the product-level feature family computable for the first time.                                                                                                                                         |
| payment_context       | IP, device, session, and address mismatch. Carries an explicit scope guard: these support per-transaction velocity only and must not be joined across accounts to build identity graphs — that is the abuse-ring direction this project does not build. |

|                  |                                                                                   |
|------------------|-----------------------------------------------------------------------------------|
| ingest_jobs      | Bulk load progress with a per-row rejection report.                               |
| webhook_events   | Keyed on the provider's own event id, which makes redelivery idempotent for free. |
| idempotency_keys | Backs the Idempotency-Key header; stores the original response for 24h.           |

## 8 · Endpoint coverage

Every group in the API reference now has storage behind it.

| API GROUP                | ENDPOINTS | BACKING TABLES                                                                |
|--------------------------|-----------|-------------------------------------------------------------------------------|
| Risk scoring             | 8         | risk_scores, risk_score_features, risk_explanations, models, threshold_config |
| Review queue             | 6         | reviews, review_decisions, review_notes, reviewers                            |
| Outcomes & labels        | 5         | risk_labels, refunds, disputes, merchants                                     |
| Models                   | 6         | models, model_feature_importance, datasets                                    |
| Evaluation               | 6         | evaluations, evaluation_points, datasets, dataset_members                     |
| Thresholds & cost        | 5         | threshold_config, cost_model_config                                           |
| Transaction data         | 16        | orders, payments, refunds, disputes, order_items                              |
| Customers                | 5         | customers, customer_stats_current, risk_scores                                |
| Bulk ingestion           | 3         | ingest_jobs, products                                                         |
| Webhooks & system        | 5         | webhook_events, models, threshold_config                                      |
| All state-changing POSTs | —         | idempotency_keys                                                              |
| All endpoints            | 65        | merchants, api_keys                                                           |

## 9 · Decisions now closed

| DECISION                 | RESOLUTION                                                                                                                                                                                                          |
|--------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Training dataset         | <b>UCI Online Retail II</b> — 1,067,371 transactions, Dec 2009 – Dec 2011, real customer returns recorded as credit notes. Binding detailed in §10.                                                                 |
| Combined vs. split label | <b>Split, by necessity.</b> No public dataset carries disputes, so label_disputed is always 0 and the modelled target reduces to label_refunded. Both columns remain, so adding disputes later needs no migration.  |
| Chargeback metrics       | <b>Not reported.</b> Synthetic dispute labels would measure the generator rather than the model, which fails the track's "honest metrics" bar. The dispute path is built and unevaluated, and the write-up says so. |
| Engine                   | SQLite for development and the demo. The DDL runs unmodified on PostgreSQL if the demo moves to Supabase; only the -- json columns become JSONB.                                                                    |

## 9b · Still open

| DECISION                         | WHY IT IS NOT SETTLED HERE                                                                                                                                                                           |
|----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Return window for is_mature      | Set it from the observed purchase-to-credit-note gap distribution once the data is loaded, not from a guess. It determines how much of the tail is excluded, and the number belongs in the write-up. |
| Minimum gap for a genuine return | Some credit notes are same-day clerical corrections rather than returns. A minimum gap (a day or more) filters them; the exact cut should follow the observed distribution.                          |

---

FP / FN cost figures

`cost_model_config` stores the formula inputs; the values must be set in GBP pence against this dataset's actual order-value distribution before the threshold sweep means anything.

---

## 10 · Dataset binding — Online Retail II

v2 was designed before a dataset was chosen. This section fixes the binding, and is the part a judge should read to understand what is real and what is absent.

The source is invoice-line shaped — one row per item per invoice, with columns Invoice, StockCode, Description, Quantity, InvoiceDate, Price, Customer ID and Country. The schema is entity shaped. The gap is closed by an ETL mapping, not by redesigning the schema: the only structural change in v3 is `orders.country`.

### 10.1 Source → table mapping

| TABLE                      | SOURCE                    | CONSTRUCTION                                                                                                                                                           |
|----------------------------|---------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| customers                  | distinct Customer ID      | first_seen_at = earliest invoice date; account_created_at null (unknown); id minted as cust_<id>.                                                                      |
| orders                     | distinct non-C Invoice    | amount = $\Sigma(\text{Quantity} \times \text{Price})$ in pence; status='paid'; receipt = invoice number; country from the source column.                              |
| order_items                | every invoice line        | sku = StockCode, unit_price, quantity, category derived from Description.                                                                                              |
| products                   | distinct StockCode        | Category derived from Description text; size_variant_type='none' — a gift retailer has no apparel sizing, so the fit-driven return mechanism is absent from this data. |
| payments                   | synthesised 1:1 per order | The source has no payment layer. One captured payment per order, carrying the order's amount and timestamp.                                                            |
| refunds                    | C-prefixed invoices       | Matched back to the original payment by customer and StockCode; refund_reason='customer_return', status='processed'.                                                   |
| risk_labels                | derived                   | label_disputed is always 0, so label_risk = label_refunded.                                                                                                            |
| datasets · dataset_members | derived                   | Chronological split on train_until, hashed once.                                                                                                                       |

### 10.2 Population status of all 30 tables

| STATUS               | COUNT | TABLES                                                                                                                                                                                                                                                      |
|----------------------|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Loaded from data     | 9     | customers, orders, order_items, products, payments, refunds, risk_labels, datasets, dataset_members                                                                                                                                                         |
| Empty by design      | 2     | disputes, payment_context                                                                                                                                                                                                                                   |
| Written at runtime   | 16    | merchants, api_keys, customer_stats_current, models, model_feature_importance, cost_model_config, evaluations, evaluation_points, threshold_config, risk_scores, risk_score_features, risk_explanations, reviewers, reviews, review_decisions, review_notes |
| Unused in this build | 3     | ingest_jobs, webhook_events, idempotency_keys — production plumbing, not needed for the demo                                                                                                                                                                |

The two empty tables are the honest record, not an omission. disputes is empty because no public dataset carries chargebacks, which is precisely why metrics are reported on returns only. payment\_context is empty because the source has no IP, device or session — so the velocity feature family is designed, storable, and unused. Leaving both tables in place and empty documents the boundary between what the system supports and what this data can evidence.

### 10.3 Field-level adaptations

| FIELD | ADAPTATION |
|-------|------------|
|-------|------------|

|                                         |                                                                                                                                                                                                                                                                   |
|-----------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Currency                                | This is a UK retailer: <code>merchants.currency='GBP'</code> and every amount is in <b>pence</b> , not paise. The FP/FN cost model in <code>cost_model_config</code> must be denominated in pence to match.                                                       |
| <code>payments.method</code>            | The column is NOT NULL and the source has no payment method. Filled with a single constant value and <b>excluded from the feature set</b> — a zero-variance feature contributes nothing, and leaving it in would imply payment-method signal that does not exist. |
| <code>orders.country</code>             | The one structural change in v3. The source's <code>Country</code> column had nowhere to go; Razorpay exposes geography on the customer or address rather than the order.                                                                                         |
| <code>products.size_variant_type</code> | Always 'none' for this catalogue. The size/fit return mechanism is real but unobservable here.                                                                                                                                                                    |

## 10.4 Label construction

For each purchase line, a positive label is a later credit note from the same customer for the same StockCode. Two filters apply, and both matter:

| FILTER           | WHY                                                                                                                                                                                                                                                                                              |
|------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Minimum time gap | Some credit notes are same-day data-entry corrections rather than returns. Without a gap requirement the model learns to predict the retailer's own clerical errors.                                                                                                                             |
| Maturity cutoff  | The data ends 9 December 2011. A purchase from early December has not had time to be returned, so it looks clean and is not. This is exactly what <code>label_matured_at</code> and <code>is_mature</code> were added for in v2 — the tail is excluded rather than silently counted as negative. |

The credit-note row itself must never enter the feature vector for the purchase it refunds — the same strictly-before discipline as Reference Query 2 in §6.

## 10.5 Feature inventory

| AVAILABLE (11)                                                                                                                                                                                                                                                                  | UNAVAILABLE IN THIS DATASET (5)                                                                                                                                 |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| order value · basket size (distinct SKUs) · total quantity · unit price vs. category mean · customer prior order count (as-of) · customer prior return rate (as-of) · account age from first-seen · hour of day · day of week · country · product-category baseline return rate | payment method (constant) · billing/shipping mismatch (no addresses) · discount depth (no coupon data) · international flag · device & IP velocity (no context) |

Eleven features is a workable set, and the two strongest predictors in return modelling — customer history and category baseline — are both present. Both surviving customer features are computed strictly as-of the transaction, per the leakage rule in §5.

## Appendix A · Complete DDL

Dependency-ordered; runs top to bottom. Verified against SQLite 3 with foreign-key enforcement on: 30 tables, 36 indexes, no errors.

```
-- =====
-- AI Risk Manager – Return & Chargeback Risk Scorer
-- Database Schema v3 (FINAL)
-- Razorpay Hackathon, Track 02
--
-- v3 finalises v2 against the chosen training dataset (UCI Online Retail II).
-- Changes from v2: orders.country added; dataset-binding notes recorded in the
-- design document (which tables load, which stay empty, and why).
--
-- Target: SQLite 3.31+ (local development) and PostgreSQL 12+ (Supabase).
--
-- Portability notes:
-- * JSON columns are declared TEXT. On PostgreSQL, change TEXT -> JSONB
--   for every column commented "-- json".
-- * BOOLEAN is stored as INTEGER 0/1 in SQLite; PostgreSQL uses it natively.
-- * All timestamps are INTEGER Unix epoch seconds, matching Razorpay's
--   public API, which returns created_at and respond_by as integers.
-- * Amounts are INTEGER in the smallest currency unit (paise for INR).
--
-- Convention: tables marked [RZP] mirror a Razorpay public API entity.
--             Tables marked [EXT] are our own additions.
-- =====

-- Tables are declared in dependency order: the file runs top-to-bottom on
-- both SQLite and PostgreSQL with no forward references.

-- =====
-- LAYER A – TENANCY
-- Every row in every other table belongs to exactly one merchant.
-- =====

-- [EXT] Merchant accounts. Every row in every other table belongs to exactly
-- one merchant; the API never accepts merchant_id as a parameter, it is
-- always derived from the API key.
CREATE TABLE merchants (
  id          TEXT PRIMARY KEY,          -- acc_XXXXXXXXXX
  name        TEXT NOT NULL,
  email       TEXT NOT NULL,
  currency    TEXT NOT NULL DEFAULT 'INR',
  dispute_window_days INTEGER NOT NULL DEFAULT 120, -- drives label maturity
  created_at  INTEGER NOT NULL
);

-- [EXT] API credentials. role gates the admin-only endpoints.
CREATE TABLE api_keys (
  id          TEXT PRIMARY KEY,          -- key_XXXXXXXXXX
  merchant_id TEXT NOT NULL REFERENCES merchants(id),
  key_hash    TEXT NOT NULL UNIQUE,      -- argon2/bcrypt, never the raw key
  role        TEXT NOT NULL CHECK (role IN ('standard','admin')),
  active      BOOLEAN NOT NULL DEFAULT 1,
  created_at  INTEGER NOT NULL,
  last_used_at INTEGER
);

-- =====
-- LAYER B – CUSTOMER IDENTITY
-- Keyed on a synthetic id, not on a phone number.
-- =====

-- [EXT] Customer identity. Keyed on a synthetic id, not on a phone number:
-- the API exposes cust_xxxx identifiers, contact details change, and PII does
-- not belong in a primary key or a URL.
CREATE TABLE customers (
  id          TEXT PRIMARY KEY,          -- cust_XXXXXXXXXX
  merchant_id TEXT NOT NULL REFERENCES merchants(id),
  email       TEXT,
  contact     TEXT,
  account_created_at INTEGER,            -- merchant's signup timestamp
  first_seen_at INTEGER NOT NULL,        -- first transaction observed
  last_seen_at INTEGER,
  created_at  INTEGER NOT NULL
);
```

```

);

-- =====
-- LAYER C – RAZORPAY-NATIVE TRANSACTION ENTITIES
-- Field sets and enum values follow Razorpay's public API objects.
-- =====

-- [RZP] Order entity.
CREATE TABLE orders (
  id TEXT PRIMARY KEY, -- order_xxxxxxxxxxx
  merchant_id TEXT NOT NULL REFERENCES merchants(id),
  amount INTEGER NOT NULL CHECK (amount >= 0),
  amount_paid INTEGER NOT NULL DEFAULT 0,
  amount_due INTEGER NOT NULL DEFAULT 0,
  currency TEXT NOT NULL DEFAULT 'INR',
  receipt TEXT, -- merchant's internal reference
  status TEXT NOT NULL CHECK (status IN ('created','attempted','paid')),
  attempts INTEGER NOT NULL DEFAULT 0,
  offer_id TEXT,
  -- [EXT] Not a Razorpay Order field. Added because the training dataset
  -- carries a country per invoice and it is a usable feature; Razorpay
  -- itself exposes geography on the customer/address, not the order.
  country TEXT,
  notes TEXT, -- json
  created_at INTEGER NOT NULL
);

-- [RZP] Payment entity. Field set follows Razorpay's documented Payment object.
CREATE TABLE payments (
  id TEXT PRIMARY KEY, -- pay_xxxxxxxxxxx
  merchant_id TEXT NOT NULL REFERENCES merchants(id),
  order_id TEXT REFERENCES orders(id),
  customer_id TEXT REFERENCES customers(id), -- resolved at ingest
  amount INTEGER NOT NULL CHECK (amount >= 0),
  currency TEXT NOT NULL DEFAULT 'INR',
  status TEXT NOT NULL CHECK (status IN
    ('created','authorized','captured','refunded','failed')),
  -- Razorpay's documented methods are card, netbanking, wallet, emi, upi.
  -- 'cod' is OUR extension: not a Razorpay method, but a first-order driver
  -- of return risk, so it is admitted here and flagged as non-native.
  method TEXT NOT NULL CHECK (method IN
    ('card','netbanking','wallet','emi','upi','cod')),
  captured BOOLEAN NOT NULL DEFAULT 0,
  international BOOLEAN NOT NULL DEFAULT 0,
  description TEXT,
  -- method-specific identifiers, populated according to `method`
  card_id TEXT,
  bank TEXT,
  wallet TEXT,
  vpa TEXT,
  acquirer_data TEXT, -- json
  -- payer contact
  email TEXT,
  contact TEXT,
  -- Razorpay's commercials
  fee INTEGER NOT NULL DEFAULT 0,
  tax INTEGER NOT NULL DEFAULT 0,
  -- failure detail
  error_code TEXT,
  error_description TEXT,
  error_source TEXT,
  error_step TEXT,
  error_reason TEXT,
  -- refund rollup
  amount_refunded INTEGER NOT NULL DEFAULT 0,
  refund_status TEXT CHECK (refund_status IS NULL
    OR refund_status IN ('partial','full')),
  notes TEXT, -- json
  created_at INTEGER NOT NULL
);

-- [RZP] Refund entity, plus one extension column.
CREATE TABLE refunds (
  id TEXT PRIMARY KEY, -- rfnd_xxxxxxxxxxx
  merchant_id TEXT NOT NULL REFERENCES merchants(id),
  payment_id TEXT NOT NULL REFERENCES payments(id),
  amount INTEGER NOT NULL CHECK (amount >= 0),
  currency TEXT NOT NULL DEFAULT 'INR',
  status TEXT NOT NULL CHECK (status IN ('pending','processed','failed')),
  speed_requested TEXT CHECK (speed_requested IS NULL
    OR speed_requested IN ('normal','optimum')),

```

```

speed_processed TEXT CHECK (speed_processed IS NULL
                           OR speed_processed IN ('normal','instant')),

receipt         TEXT,
batch_id        TEXT,
notes           TEXT,
-- json
-- [EXT] Razorpay carries no refund reason, but the distinction is decisive:
-- a merchant-side refund (out of stock, pricing error) is NOT a return-risk
-- event and must not enter the positive class. Set at ingest or via
-- POST /outcomes.
refund_reason   TEXT CHECK (refund_reason IS NULL OR refund_reason IN
                           ('customer_return',    -- counts toward label_refunded
                            'customer_cancellation', -- counts
                            'merchant_stock_out',   -- excluded
                            'merchant_error',       -- excluded
                            'duplicate_charge',    -- excluded
                            'goodwill',            -- excluded
                            'other')),
created_at      INTEGER NOT NULL
);

-- [RZP] Dispute entity ("chargeback"). Enum values follow Razorpay exactly.
CREATE TABLE disputes (
  id TEXT PRIMARY KEY, -- disp_XXXXXXXXXXXX
  merchant_id TEXT NOT NULL REFERENCES merchants(id),
  payment_id TEXT NOT NULL REFERENCES payments(id),
  amount INTEGER NOT NULL CHECK (amount >= 0),
  currency TEXT NOT NULL DEFAULT 'INR',
  -- amount actually removed from the merchant balance when a dispute is lost;
  -- this, not `amount`, is the true false-negative cost input.
  amount_deducted INTEGER NOT NULL DEFAULT 0,
  reason_code TEXT NOT NULL,
  reason_description TEXT,
  respond_by INTEGER, -- epoch deadline for evidence
  status TEXT NOT NULL CHECK (status IN
                              ('open','under_review','won','lost','closed')),
  -- 'fraud' and 'retrieval' are Razorpay phases that are OUT OF SCOPE for
  -- this project: fraud is third-party identity theft (a different track),
  -- retrieval is an information request, not a loss. They are stored so they
  -- can be excluded explicitly rather than silently absorbed into the label.
  phase TEXT NOT NULL CHECK (phase IN
                              ('fraud','retrieval','chargeback','pre_arbitration','arbitration')),
  evidence TEXT, -- json
  created_at INTEGER NOT NULL
);

-- =====
-- LAYER D – COMMERCE DETAIL
-- Product-level and velocity features live here.
-- =====

-- [EXT] Product catalogue. Backs the product-level feature family:
-- category baseline return rate, price relative to category average,
-- size/fit dependency.
CREATE TABLE products (
  id TEXT PRIMARY KEY, -- prod_XXXXXXXXXXXX
  merchant_id TEXT NOT NULL REFERENCES merchants(id),
  sku TEXT NOT NULL,
  name TEXT,
  category TEXT NOT NULL,
  subcategory TEXT,
  list_price INTEGER NOT NULL CHECK (list_price >= 0),
  -- how strongly the item's fit drives returns; apparel/footwear dominate
  size_variant_type TEXT NOT NULL DEFAULT 'none'
  CHECK (size_variant_type IN ('none','apparel','footwear','other')),
  created_at INTEGER NOT NULL,
  UNIQUE (merchant_id, sku)
);

-- [EXT] Line items on an order.
CREATE TABLE order_items (
  id TEXT PRIMARY KEY, -- oitem_XXXXXXXXXXXX
  order_id TEXT NOT NULL REFERENCES orders(id),
  product_id TEXT REFERENCES products(id),
  sku TEXT NOT NULL,
  category TEXT, -- denormalised for cold-start
  unit_price INTEGER NOT NULL CHECK (unit_price >= 0),
  quantity INTEGER NOT NULL CHECK (quantity > 0),
  size_variant TEXT,
  discount_applied INTEGER NOT NULL DEFAULT 0
);

```



```

-- [EXT] Per-transaction context for velocity features only.
-- SCOPE GUARD: these columns support "how many attempts from this device in
-- the last hour" (a per-transaction velocity signal). They must NOT be joined
-- across accounts to build identity graphs – that is the abuse-ring direction,
-- which this project explicitly does not build.
CREATE TABLE payment_context (
  payment_id      TEXT PRIMARY KEY REFERENCES payments(id),
  ip              TEXT,
  device_id       TEXT,
  session_id      TEXT,
  user_agent       TEXT,
  billing_pincode TEXT,
  shipping_pincode TEXT,
  address_mismatch BOOLEAN,
  created_at      INTEGER NOT NULL
);

-- =====
-- LAYER E – SERVING-TIME AGGREGATES
-- Mutable cache. Never read at training time. See the leakage rule below.
-- =====

-- [EXT] SERVING-TIME AGGREGATE CACHE – NOT A TRAINING SOURCE.
--
-- This table holds one mutable row per customer, refreshed as outcomes land.
-- `as_of_ts` records the moment the counters are valid for, and exists so that
-- the table can never be mistaken for a timeless fact.
--
-- LEAKAGE RULE: training and evaluation MUST NOT read this table. A row read
-- at training time carries the customer's FINAL return rate, which already
-- contains the outcome of the very payment being predicted plus every later
-- outcome. A chronological split on payments.created_at does not catch this,
-- because the leak arrives through an aggregate that has no time dimension.
-- Training reads risk_score_features (frozen at scoring time) or recomputes
-- as-of values from refunds/disputes filtered to created_at < the payment's
-- created_at. See the reference query at the foot of this file.
CREATE TABLE customer_stats_current (
  customer_id      TEXT PRIMARY KEY REFERENCES customers(id),
  merchant_id      TEXT NOT NULL REFERENCES merchants(id),
  as_of_ts         INTEGER NOT NULL,      -- counters are true as of this epoch
  total_orders     INTEGER NOT NULL DEFAULT 0,
  total_payments_captured INTEGER NOT NULL DEFAULT 0,
  total_refunds    INTEGER NOT NULL DEFAULT 0,
  total_returns    INTEGER NOT NULL DEFAULT 0,  -- refunds that are returns
  total_disputes   INTEGER NOT NULL DEFAULT 0,  -- in-scope phases only
  -- denominators are captured payments, not orders: disputes attach to
  -- payments, and one order can carry several attempts.
  past_return_rate REAL NOT NULL DEFAULT 0,
  past_dispute_rate REAL NOT NULL DEFAULT 0,
  computed_at      INTEGER NOT NULL
);

-- =====
-- LAYER F – LABELS & DATASETS
-- The training target, with maturity and exclusions made explicit.
-- =====

-- [EXT] The training target, with maturity and exclusions made explicit.
--
-- Two things v1 could not express and this table now can:
-- * MATURITY – a payment with no dispute row is not "clean" until its
--   dispute window has closed. Without is_mature, a model trained on recent
--   data learns that recent orders are safe, and held-out metrics flatter.
-- * EXCLUSIONS – fraud-phase disputes, retrieval requests and merchant-side
--   refunds are not return/chargeback risk. They are recorded and excluded,
--   not silently counted as positives.
CREATE TABLE risk_labels (
  payment_id      TEXT PRIMARY KEY REFERENCES payments(id),
  merchant_id     TEXT NOT NULL REFERENCES merchants(id),
  label_refunded  BOOLEAN NOT NULL DEFAULT 0,  -- customer-driven refunds only
  label_disputed  BOOLEAN NOT NULL DEFAULT 0,  -- in-scope dispute phases only
  label_risk      BOOLEAN NOT NULL DEFAULT 0,  -- refunded OR disputed
  outcome_observed_at INTEGER,                -- when the outcome landed, if any
  label_matured_at INTEGER NOT NULL,          -- payment.created_at + window
  is_mature       BOOLEAN NOT NULL DEFAULT 0,  -- label_matured_at <= now
  excluded_reason TEXT CHECK (excluded_reason IS NULL OR excluded_reason IN
    ('fraud_dispute','retrieval_request','merchant_refund',
     'immature','payment_not_captured')),
  label_version   INTEGER NOT NULL DEFAULT 1,
  derived_at      INTEGER NOT NULL
);

```

```

);

-- [EXT] A frozen train/test split. Written once, hashed, and referenced by
-- every model and evaluation, so a re-run can never silently re-split.
CREATE TABLE datasets (
  id          TEXT PRIMARY KEY,          -- ds_XXXXXXXXXXXX
  merchant_id TEXT NOT NULL REFERENCES merchants(id),
  name        TEXT NOT NULL,
  train_until INTEGER NOT NULL,          -- chronological cut point, epoch
  exclude_immature BOOLEAN NOT NULL DEFAULT 1,
  n_train     INTEGER NOT NULL DEFAULT 0,
  n_test      INTEGER NOT NULL DEFAULT 0,
  positive_rate REAL,
  split_hash  TEXT NOT NULL,            -- hash of the member set
  created_at  INTEGER NOT NULL
);

CREATE TABLE dataset_members (
  dataset_id TEXT NOT NULL REFERENCES datasets(id),
  payment_id TEXT NOT NULL REFERENCES payments(id),
  split      TEXT NOT NULL CHECK (split IN ('train','test')),
  PRIMARY KEY (dataset_id, payment_id)
);

-- =====
-- LAYER G – MODELS, COST MODEL & EVALUATION
-- =====

CREATE TABLE models (
  id          TEXT PRIMARY KEY,          -- mdl_XXXXXXXXXXXX
  merchant_id TEXT NOT NULL REFERENCES merchants(id),
  algorithm    TEXT NOT NULL CHECK (algorithm IN
    ('logistic_regression','lightgbm','xgboost')),
  dataset_id  TEXT NOT NULL REFERENCES datasets(id),
  hyperparameters TEXT,                -- json
  feature_list TEXT NOT NULL,           -- json array, ordered
  train_until INTEGER NOT NULL,
  status      TEXT NOT NULL CHECK (status IN
    ('training','trained','active','archived','failed')),
  artifact_path TEXT,                  -- joblib/pickle location
  trained_at  INTEGER,
  activated_at INTEGER,
  created_at  INTEGER NOT NULL
);

CREATE TABLE model_feature_importance (
  model_id    TEXT NOT NULL REFERENCES models(id),
  feature_name TEXT NOT NULL,
  importance  REAL NOT NULL,            -- coefficient or gain
  rank        INTEGER NOT NULL,
  PRIMARY KEY (model_id, feature_name)
);

-- [EXT] False-positive and false-negative costs, stored as the formula inputs
-- rather than a bare number, so the figure can be audited instead of trusted.
CREATE TABLE cost_model_config (
  id          TEXT PRIMARY KEY,          -- cost_XXXXXXXXXXXX
  merchant_id TEXT NOT NULL REFERENCES merchants(id),
  currency    TEXT NOT NULL DEFAULT 'INR',
  -- false positive: a genuine customer is held or friction'd
  lost_sale_probability REAL NOT NULL,
  average_order_value  INTEGER NOT NULL,
  trust_penalty        INTEGER NOT NULL,
  cost_per_fp          INTEGER NOT NULL,  -- p*AOV + trust_penalty
  -- false negative: a real return or chargeback is missed
  refunded_amount      INTEGER NOT NULL,
  chargeback_fee       INTEGER NOT NULL,
  logistics_restocking INTEGER NOT NULL,
  cost_per_fn          INTEGER NOT NULL,
  reason              TEXT NOT NULL,      -- why this version exists
  created_by          TEXT,
  is_current          BOOLEAN NOT NULL DEFAULT 0,
  created_at          INTEGER NOT NULL
);

-- [EXT] One held-out evaluation run. Records the cost model in force at run
-- time, so an old run can never be reinterpreted under new cost assumptions.
CREATE TABLE evaluations (
  id          TEXT PRIMARY KEY,          -- eval_XXXXXXXXXXXX
  merchant_id TEXT NOT NULL REFERENCES merchants(id),
  model_id    TEXT NOT NULL REFERENCES models(id),

```

```

dataset_id      TEXT      NOT NULL REFERENCES datasets(id),
cost_model_id   TEXT      NOT NULL REFERENCES cost_model_config(id),
n_samples       INTEGER,
base_rate       REAL,           -- positive rate in the test slice
threshold_evaluated REAL,
precision_score REAL,
recall_score    REAL,
f1_score        REAL,
pr_auc         REAL,
roc_auc        REAL,
tp INTEGER, fp INTEGER, fn INTEGER, tn INTEGER,
total_cost      INTEGER,
optimal_threshold REAL,
status         TEXT      NOT NULL CHECK (status IN ('running','completed','failed')),
started_at      INTEGER NOT NULL,
completed_at    INTEGER
);

-- [EXT] The threshold sweep: one row per threshold step. This is the series
-- behind the cost-vs-threshold chart, and evaluations.optimal_threshold is the
-- argmin of total_cost over these rows – not the accuracy-maximising point.
CREATE TABLE evaluation_points (
  eval_id      TEXT      NOT NULL REFERENCES evaluations(id),
  threshold    REAL      NOT NULL,
  tp INTEGER NOT NULL, fp INTEGER NOT NULL,
  fn INTEGER NOT NULL, tn INTEGER NOT NULL,
  precision_score REAL,
  recall_score  REAL,
  total_cost    INTEGER NOT NULL,
  PRIMARY KEY (eval_id, threshold)
);

-- =====
-- LAYER H – THRESHOLD CONFIGURATION
-- Versioned; links back to the evaluation run that produced the cut points.
-- =====

-- [EXT] Band cut points. source='optimized' links back to the evaluation run
-- whose cost curve produced them.
CREATE TABLE threshold_config (
  id          TEXT PRIMARY KEY,           -- thr_XXXXXXXXXX
  merchant_id TEXT      NOT NULL REFERENCES merchants(id),
  threshold_low REAL    NOT NULL CHECK (threshold_low BETWEEN 0 AND 1),
  threshold_high REAL    NOT NULL CHECK (threshold_high BETWEEN 0 AND 1),
  source       TEXT      NOT NULL CHECK (source IN ('manual','optimized')),
  eval_id      TEXT      REFERENCES evaluations(id),
  reason       TEXT      NOT NULL,
  created_by   TEXT,
  is_current   BOOLEAN NOT NULL DEFAULT 0,
  created_at   INTEGER NOT NULL,
  CHECK (threshold_low <= threshold_high)
);

-- =====
-- LAYER I – SCORING
-- =====

-- [EXT] One row per score produced. Backs POST /risk/score and /risk/scores/*.
CREATE TABLE risk_scores (
  id          TEXT PRIMARY KEY,           -- score_XXXXXXXXXX
  merchant_id TEXT      NOT NULL REFERENCES merchants(id),
  payment_id  TEXT      REFERENCES payments(id),
  order_id    TEXT      REFERENCES orders(id),
  customer_id TEXT      REFERENCES customers(id),
  model_id    TEXT      NOT NULL REFERENCES models(id),
  risk_probability REAL    NOT NULL CHECK (risk_probability BETWEEN 0 AND 1),
  risk_band   TEXT      NOT NULL CHECK (risk_band IN ('low','medium','high')),
  threshold_applied REAL    NOT NULL,
  threshold_config_id TEXT NOT NULL REFERENCES threshold_config(id),
  recommendation TEXT      NOT NULL CHECK (recommendation IN
    ('allow','manual_review','hold_payout','request_verification')),
  customer_history TEXT      NOT NULL CHECK (customer_history IN ('present','none')),
  review_id   TEXT,           -- set when the score opens a review
  scoring_latency_ms INTEGER,
  created_at  INTEGER NOT NULL
);

-- [EXT] THE AS-OF FEATURE SNAPSHOT.
--
-- The exact feature vector the model saw, frozen at scoring time, with each

```

```

-- feature's contribution to the score. This table does three jobs at once:
-- 1. makes a score reproducible months later, when the aggregates have moved;
-- 2. supplies GET /risk/scores/{id}/features;
-- 3. is the leakage-free training source – every value here was computed
--    from data that existed strictly before the transaction.
CREATE TABLE risk_score_features (
  score_id      TEXT      NOT NULL REFERENCES risk_scores(id),
  feature_name  TEXT      NOT NULL,
  feature_value REAL,
  contribution  REAL,      -- signed contribution to the score
  PRIMARY KEY (score_id, feature_name)
);

-- [EXT] LLM output. Read-only with respect to the score: there is no column
-- here that can alter risk_probability, risk_band or recommendation.
-- risk_probability_at_generation copies the score the text was written against,
-- so a stale explanation can be detected rather than trusted.
CREATE TABLE risk_explanations (
  score_id      TEXT PRIMARY KEY REFERENCES risk_scores(id),
  status        TEXT      NOT NULL CHECK (status IN ('pending','ready','failed')),
  summary       TEXT,
  risk_probability_at_generation REAL,
  generated_by  TEXT,      -- model identifier
  prompt_version TEXT,
  error_message TEXT,
  requested_at  INTEGER NOT NULL,
  generated_at  INTEGER
);

-- =====
-- LAYER J – HUMAN REVIEW
-- The only path by which a score becomes an action.
-- =====

-- [EXT] People who may decide. A decision without a row here is impossible.
CREATE TABLE reviewers (
  id            TEXT PRIMARY KEY,      -- usr_XXXXXXXXXXXX
  merchant_id   TEXT      NOT NULL REFERENCES merchants(id),
  name          TEXT      NOT NULL,
  email         TEXT      NOT NULL,
  active        BOOLEAN NOT NULL DEFAULT 1,
  created_at    INTEGER NOT NULL
);

-- [EXT] Review queue item, opened when a score lands in a flagged band.
CREATE TABLE reviews (
  id            TEXT PRIMARY KEY,      -- rev_XXXXXXXXXXXX
  merchant_id   TEXT      NOT NULL REFERENCES merchants(id),
  score_id      TEXT      NOT NULL REFERENCES risk_scores(id),
  payment_id    TEXT      REFERENCES payments(id),
  status        TEXT      NOT NULL CHECK (status IN ('open','assigned','closed')),
  assignee_id   TEXT      REFERENCES reviewers(id),
  opened_at     INTEGER NOT NULL,
  assigned_at   INTEGER,
  closed_at     INTEGER
);

-- [EXT] APPEND-ONLY decision log. A reversal is a new row on a reopened
-- review, never an update. reviewer_id is NOT NULL by design: there is no
-- system reviewer, and neither the model nor the LLM can write here.
CREATE TABLE review_decisions (
  id            TEXT PRIMARY KEY,
  review_id     TEXT      NOT NULL REFERENCES reviews(id),
  reviewer_id   TEXT      NOT NULL REFERENCES reviewers(id),
  decision      TEXT      NOT NULL CHECK (decision IN
    ('approve','hold_payout','request_verification','cancel_order')),
  reason        TEXT,
  agreed_with_model BOOLEAN,
  created_at    INTEGER NOT NULL
);

CREATE TABLE review_notes (
  id            TEXT PRIMARY KEY,
  review_id     TEXT      NOT NULL REFERENCES reviews(id),
  reviewer_id   TEXT      NOT NULL REFERENCES reviewers(id),
  note          TEXT      NOT NULL,
  created_at    INTEGER NOT NULL
);

-- =====

```

```

-- LAYER K - OPERATIONS
-- =====

CREATE TABLE ingest_jobs (
  id          TEXT PRIMARY KEY,          -- job_XXXXXXXXXXXX
  merchant_id TEXT NOT NULL REFERENCES merchants(id),
  file_name   TEXT,
  entity_type TEXT CHECK (entity_type IN
    ('orders', 'payments', 'refunds', 'disputes', 'products', 'mixed')),
  status      TEXT NOT NULL CHECK (status IN
    ('queued', 'running', 'completed', 'failed')),
  rows_total  INTEGER NOT NULL DEFAULT 0,
  rows_accepted INTEGER NOT NULL DEFAULT 0,
  rows_rejected INTEGER NOT NULL DEFAULT 0,
  rejections  TEXT,                    -- json, per-row reasons
  started_at  INTEGER NOT NULL,
  completed_at INTEGER
);

-- [EXT] Inbound Razorpay webhooks. The primary key is the provider's event id,
-- which makes redelivery idempotent for free.
CREATE TABLE webhook_events (
  id          TEXT PRIMARY KEY,          -- provider event id
  merchant_id TEXT NOT NULL REFERENCES merchants(id),
  event_type  TEXT NOT NULL,            -- payment.captured, refund.created, ...
  payload     TEXT NOT NULL,            -- json
  signature_verified BOOLEAN NOT NULL DEFAULT 0,
  processing_status TEXT NOT NULL CHECK (processing_status IN
    ('received', 'processed', 'failed', 'ignored')),
  error_message TEXT,
  received_at  INTEGER NOT NULL,
  processed_at INTEGER
);

-- [EXT] Backs the Idempotency-Key header on every state-changing POST.
CREATE TABLE idempotency_keys (
  key          TEXT NOT NULL,
  merchant_id  TEXT NOT NULL REFERENCES merchants(id),
  endpoint     TEXT NOT NULL,
  request_hash TEXT NOT NULL,
  response_status INTEGER,
  response_body TEXT,                  -- json
  created_at   INTEGER NOT NULL,
  expires_at   INTEGER NOT NULL,      -- created_at + 24h
  PRIMARY KEY (merchant_id, key)
);

-- =====
-- INDEXES
-- Without these the velocity features degrade to full table scans: they are
-- per-customer range queries over created_at.
-- =====

CREATE INDEX idx_orders_merchant_created ON orders (merchant_id, created_at);
CREATE INDEX idx_orders_receipt          ON orders (merchant_id, receipt);

CREATE INDEX idx_payments_order          ON payments (order_id);
CREATE INDEX idx_payments_customer_created ON payments (customer_id, created_at);
CREATE INDEX idx_payments_merchant_created ON payments (merchant_id, created_at);
CREATE INDEX idx_payments_email          ON payments (merchant_id, email);
CREATE INDEX idx_payments_contact        ON payments (merchant_id, contact);
CREATE INDEX idx_payments_status_method  ON payments (merchant_id, status, method);

CREATE INDEX idx_refunds_payment         ON refunds (payment_id);
CREATE INDEX idx_refunds_created         ON refunds (merchant_id, created_at);

CREATE INDEX idx_disputes_payment        ON disputes (payment_id);
CREATE INDEX idx_disputes_created        ON disputes (merchant_id, created_at);
CREATE INDEX idx_disputes_phase_status   ON disputes (merchant_id, phase, status);

CREATE INDEX idx_order_items_order       ON order_items (order_id);
CREATE INDEX idx_order_items_product     ON order_items (product_id);
CREATE INDEX idx_products_category       ON products (merchant_id, category);

CREATE INDEX idx_ctx_device              ON payment_context (device_id, created_at);
CREATE INDEX idx_ctx_ip                  ON payment_context (ip, created_at);

CREATE INDEX idx_customers_email         ON customers (merchant_id, email);
CREATE INDEX idx_customers_contact       ON customers (merchant_id, contact);

CREATE INDEX idx_scores_payment          ON risk_scores (payment_id, created_at);

```

```

CREATE INDEX idx_scores_merchant_created ON risk_scores (merchant_id, created_at);
CREATE INDEX idx_scores_band ON risk_scores (merchant_id, risk_band, created_at);
CREATE INDEX idx_scores_model ON risk_scores (model_id);

CREATE INDEX idx_reviews_status ON reviews (merchant_id, status, opened_at);
CREATE INDEX idx_reviews_assignee ON reviews (assignee_id, status);
CREATE INDEX idx_review_decisions_review ON review_decisions (review_id, created_at);

CREATE INDEX idx_labels_risk ON risk_labels (merchant_id, label_risk, is_mature);
CREATE INDEX idx_labels_matured ON risk_labels (label_matured_at);

CREATE INDEX idx_dataset_members_split ON dataset_members (dataset_id, split);
CREATE INDEX idx_eval_points_cost ON evaluation_points (eval_id, total_cost);
CREATE INDEX idx_webhook_status ON webhook_events (merchant_id, processing_status);
CREATE INDEX idx_idem_expiry ON idempotency_keys (expires_at);

-- Exactly one active model per merchant.
CREATE UNIQUE INDEX idx_models_one_active
ON models (merchant_id) WHERE status = 'active';

-- Exactly one current config row per merchant, per config type.
CREATE UNIQUE INDEX idx_costmodel_one_current
ON cost_model_config (merchant_id) WHERE is_current = 1;
CREATE UNIQUE INDEX idx_threshold_one_current
ON threshold_config (merchant_id) WHERE is_current = 1;

```

AI Risk Manager — Return & Chargeback Risk Scorer · Razorpay Hackathon Track 02 · Database Schema v3 (final) · Companion to [AI\\_Risk\\_Manager\\_API\\_Reference.pdf](#) and [AI\\_Risk\\_Manager\\_Approach.pdf](#). Razorpay entity fields verified against the published Payments and Disputes entity documentation.